一、从 C 语言到 PE 文件

下面是一段非常简单的 C 语言代码，几乎所有人的第一行代码都是这个：

```
#include <stdio.h>

int main() {
    printf_s("hello,world");
};
```

但是，这段代码说白了也只是纯文本，而计算机系统是只认二进制数据的，C 语言是怎么从纯文本变成可执行文件的呢。答案是通过编译器。

在 C 代码变成 exe 文件的过程中，编译器做了如下的事情：



也就是说，等到 C 语言变成 exe 文件的时候，C 语言原本的样子已经面目全非了，如果你使用十六进制编辑器去打开一个 exe 文件，你是绝对不可能从中直接还原出 C 代码的。

但是，PE 文件是一个有逻辑的可执行程序而不是不可逆加密算法，也就是说，也许很难，但 PE 文件一定是可逆的。这就是逆向工程的意义。

二、计算机发展史

2.1 电信号

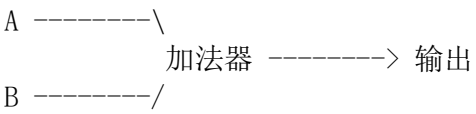
对于 CPU 来说，它不认识这些高级语言，这个物理元件只认识电压，那时候，规定高电压为 1 低电压为 0，CPU 通过电信号控制晶体管的导通与截止，进而执行与或非等运算

2.2 机器码

先从最早的电子电路讲起，假设世界上只有一个功能：

两个数相加

你完全可以设计一块电路：



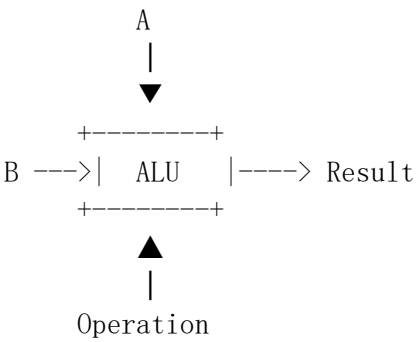
CPU 上电之后，只会做一件事情：加法。没有任何问题，但是它只能永远做加法。

如果想做减法呢？就得再设计一块减法器；如果想做乘法？再设计乘法器；如果想跳转？重新设计。

你会发现每增加一个功能，就要重新造一块电路。这显然不现实。

于是人们想到：能不能让一块电路干很多事情？

例如设计一个 ALU（算术逻辑单元）：



这里多了一个东西：

Operation

它告诉 ALU：

今天干什么。

例如：

```
00 -> ADD
01 -> SUB
10 -> XOR
11 -> AND
```

这里其实已经出现了机器码的思想。不是直接控制每一个晶体管。而是给电路一个”操作编号”。**机器码就是”操作编号”**

这里的机器码也是计算机系统能唯一读懂的编码，所有的高级语言最后都必须变成机器码才能让计算机系统读懂。它是软件和硬件间的一种“控制语言”。

2.3 汇编语言

尽管机器码作为软硬件间的桥梁已经让我们无需去物理层控制物理元件，但是对于人类来说，还是过于抽象。于是有人想，为什么不用机器码的实际意思，创造一些“助记符”，比如：0x31 表示 xor(异或)运算，为什么不干脆让人类编写程序时直接写 xor，而不是写 0x31，在中间加一层“翻译层”，最终将 xor 翻译成 0x31。于是，汇编语言诞生了，**它本质是机器码的“助记符”**，因此，汇编语言也是最接近底层的语言。

2.4 高级语言

尽管汇编语言已经足够方便，但是还是过于晦涩难懂，于是有人又在汇编的基础上封装了一层，今天所用的各种高级语言就诞生了。

三、IDA 的使用

讲了这么多，我们直接用一个例子来串联一下，用 IDA 打开 hello.exe。

3.1 窗口

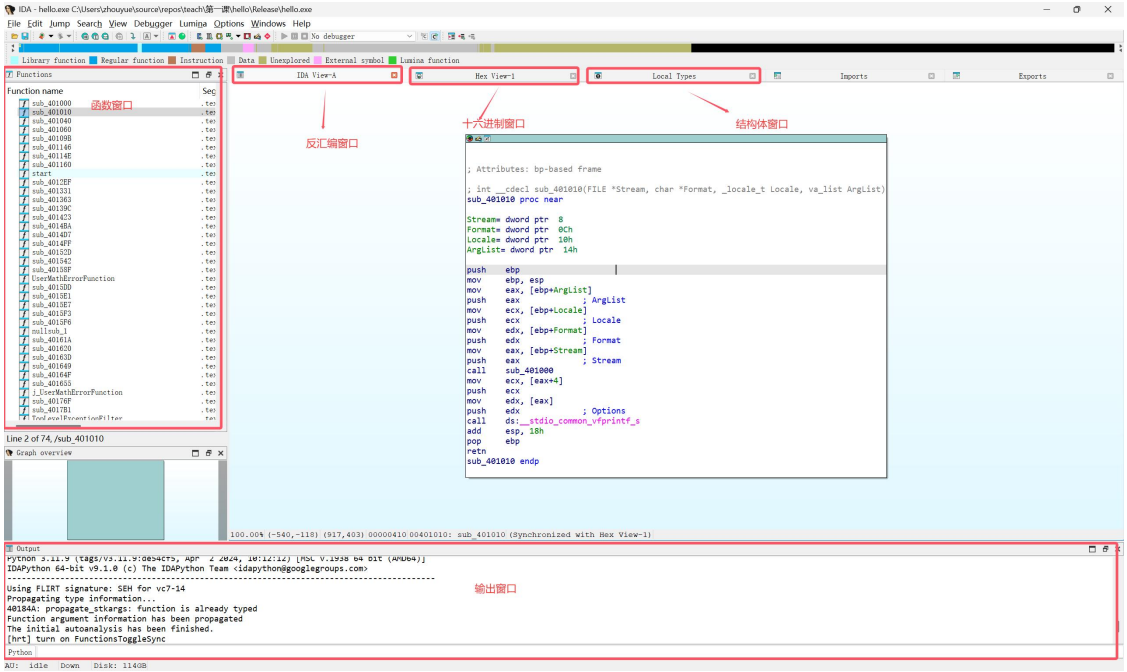


image-20260705191617640

其中，最重要的就是反汇编窗口、函数窗口和结构体窗口。

在反汇编窗口中按 F5，即可生成对应伪 C 代码窗口：

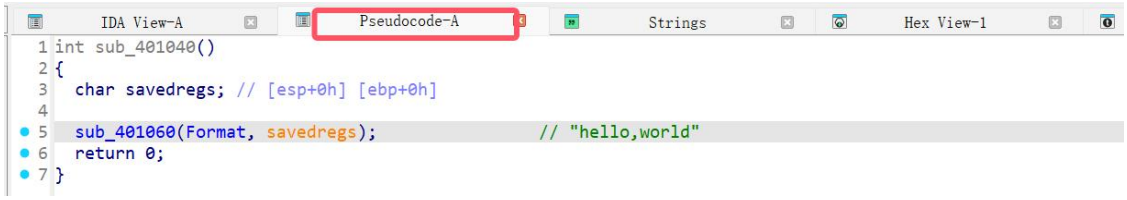


image-20260705191858787

shift + F12，即可调出字符串窗口：

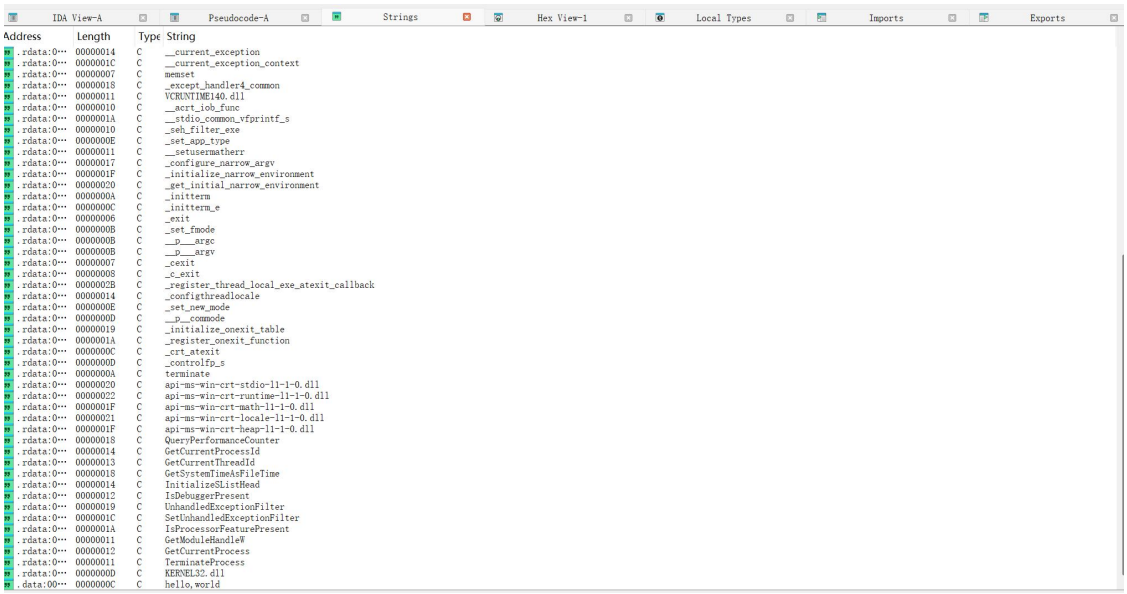


image-20260705192200007

3.2 基础操作

窗口同步

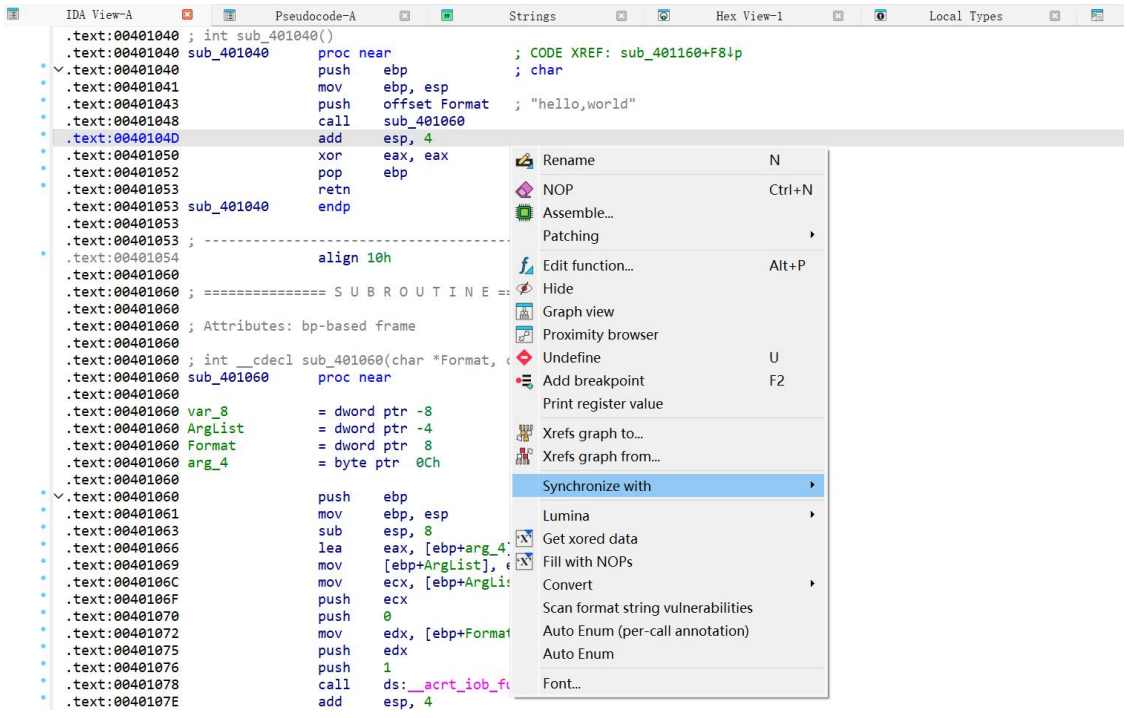


image-20260705192025167

交叉引用

数据、参数、函数、变量都可以选中后按”X”进行交叉引用，查找该对象在哪里被引用

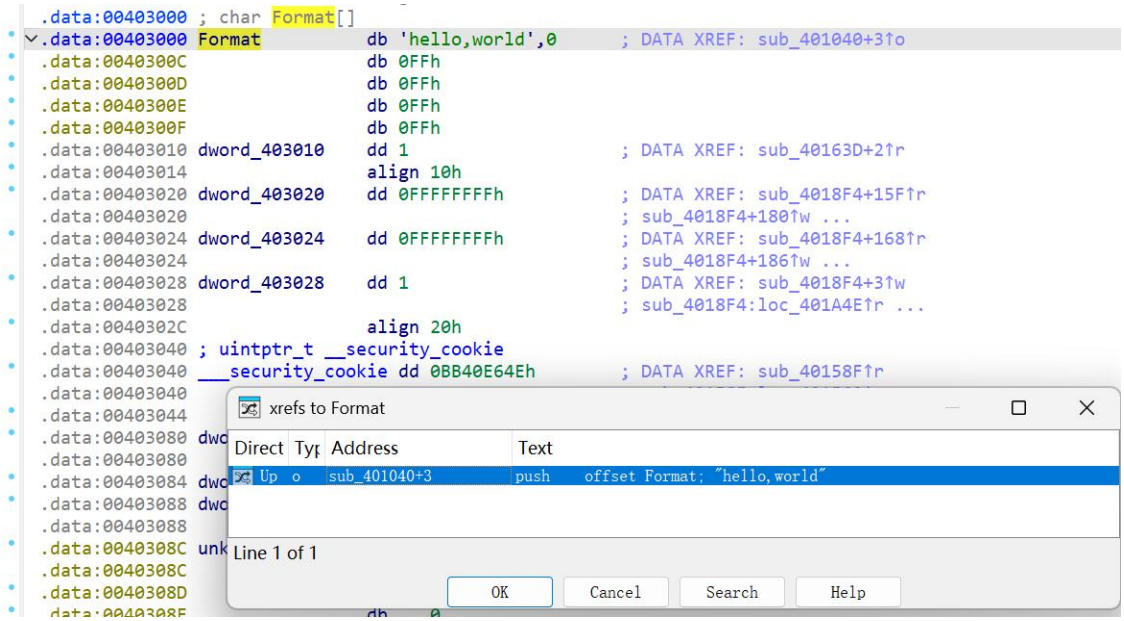


image-20260705192408879

地址跳转

按“G”可以跳转至任意地址

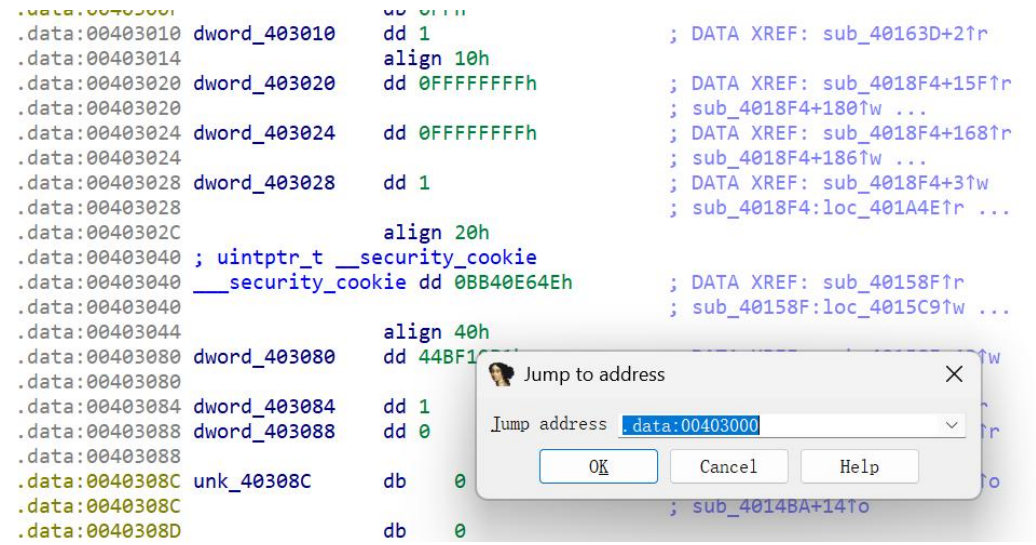


image-20260705192436070

书写注释

在伪代码窗口任意一行按下 / 键，即可输入或修改注释。

在反编译窗口任意一条汇编语句上按下 ； 键，也可以输入或修改注释。

修改符号名

光标放在某个变量或函数名上按 N 键可以修改其名字。

数据类型操作

IDA 开发了一套类型分析系统，用来处理各种数据类型（函数声明、变量声明、结构体声明等），并且允许用户自由指定。选中变量、函数后按 Y 键，弹出“Please enter the type declaration” 对话框，从中输入正确的 C 语言类型，IDA 就可以解析并自动应用这个类型。

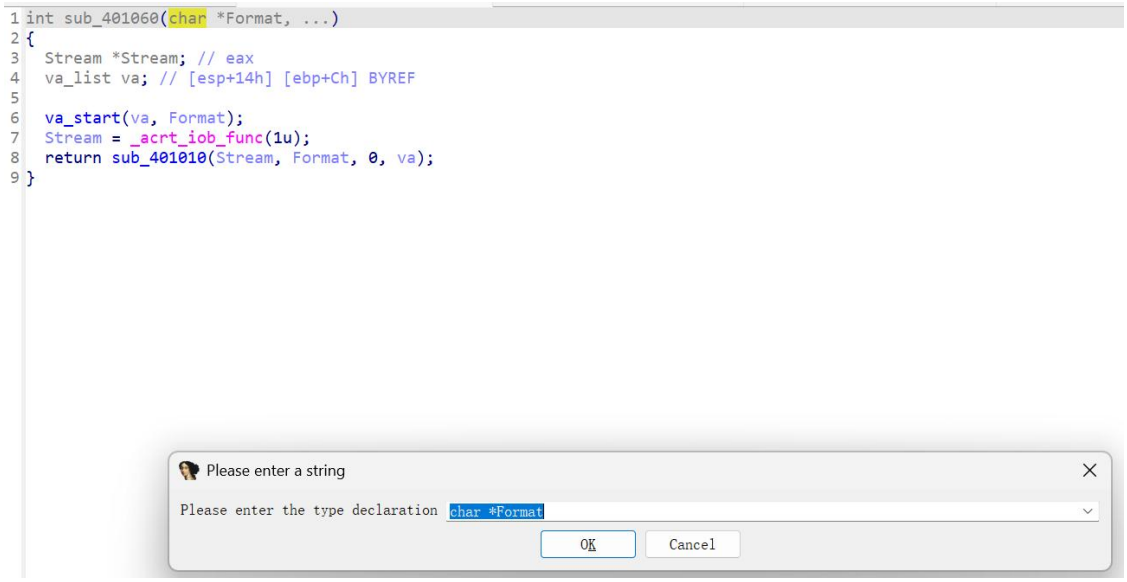


image-20260705192646817

修改数据长度

在同一个地址处多次按下 D 键，可以让数据在 1、2、4、8 字节长度间转换。

修改数据

在汇编窗口右键->Patching->Change byte:

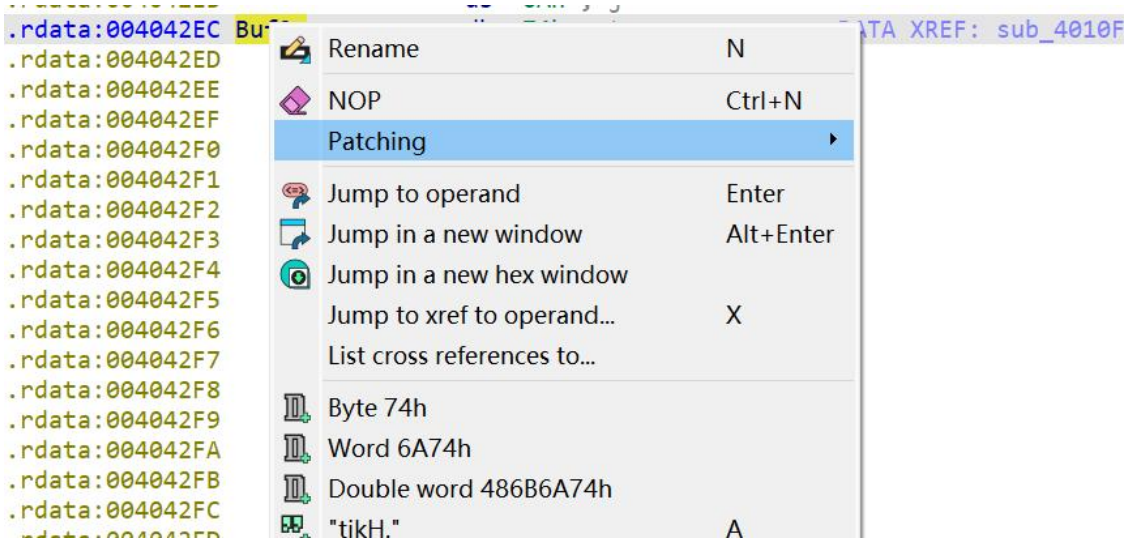


image-20260706204133227

在这个窗口可以直接使用 16 进制进行更改：

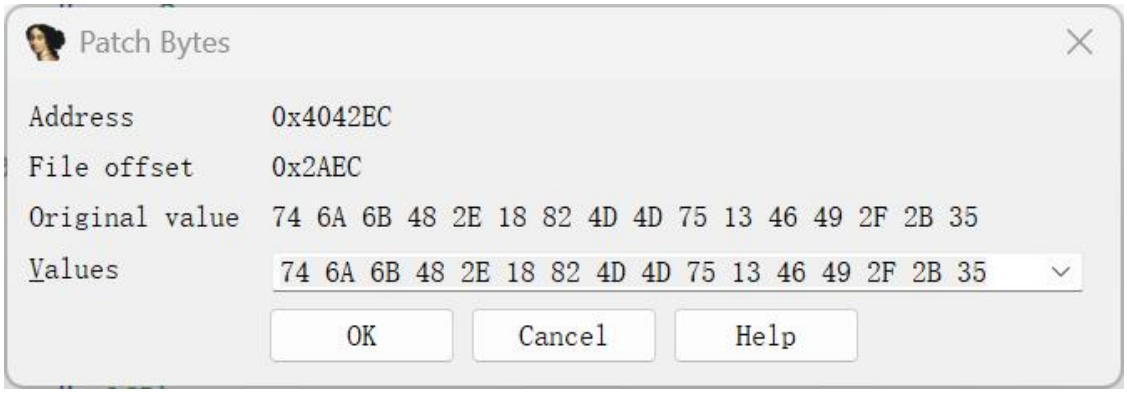


image-20260706204259495

函数操作

定义函数：在反汇编窗口选中对应行后，按 P 键

删除函数：函数窗口中选中函数后，按 Delete 键，或光标放在函数头按 U

修改函数范围：在函数窗口中选中并按 Ctrl+E 组合键，或在反汇编窗口的函数内部按 Alt+P 组合键

代码和数据的转换

U 键：取消一个地方已有的数据

D 键：让某一个位置变成数据

C 键：让某一个位置变成指令

A 键：会以该位置为起点定义一个以 \0 结尾的字符串类型

O 键：将此处定义为一个地址偏移

创建数组

按 * 键将光标处定义为一个数组，可以修改数组长度、修改显示形式等。

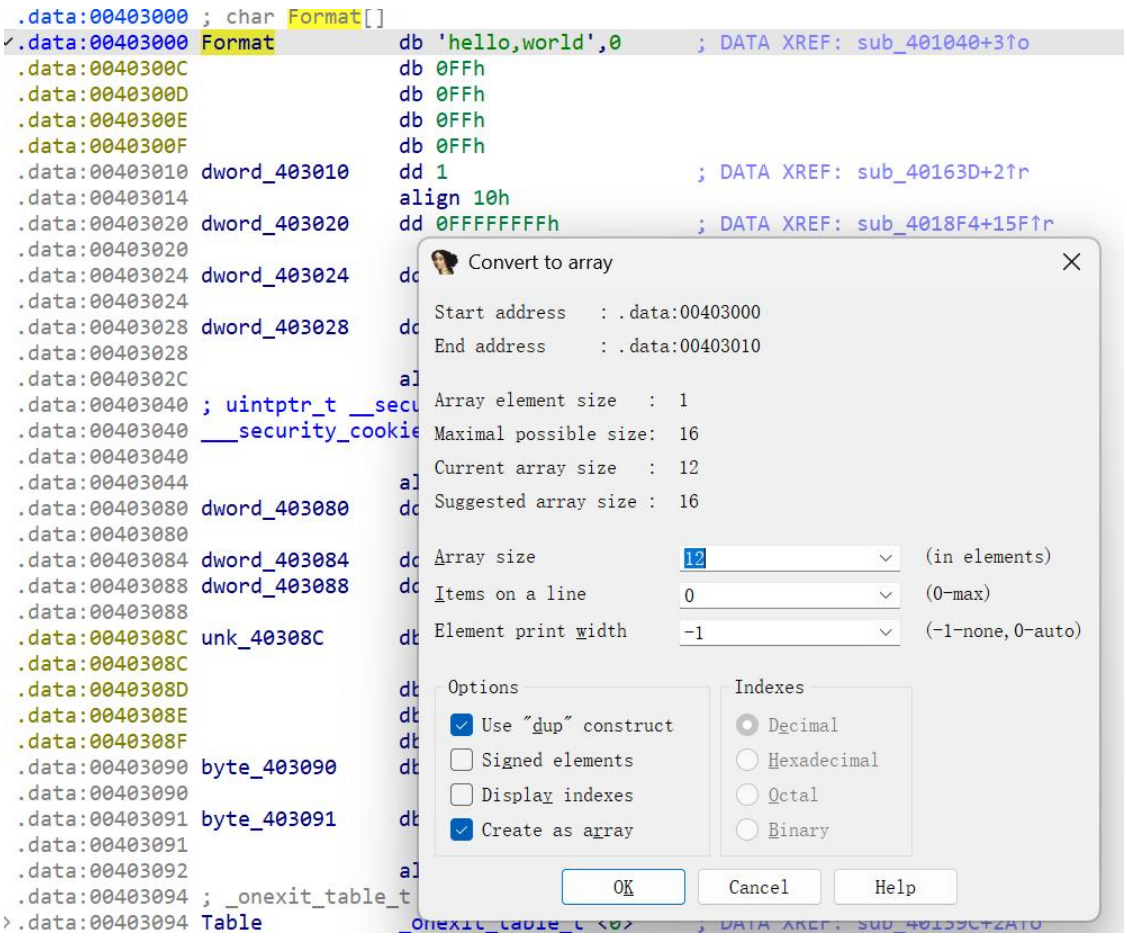


image-20260705192803241

创建结构体

我们一般更倾向于按 Shift+F1 打开 Local Types 窗口，在这里用 C 代码的形式定义结构体。尽管 IDA 中可以编辑自定义结构体，但是我一般习惯导入 C 语言文件头。

提取数据

先选出需要提取的选取，按下 Shift+E 键，并选择合适的格式导出。

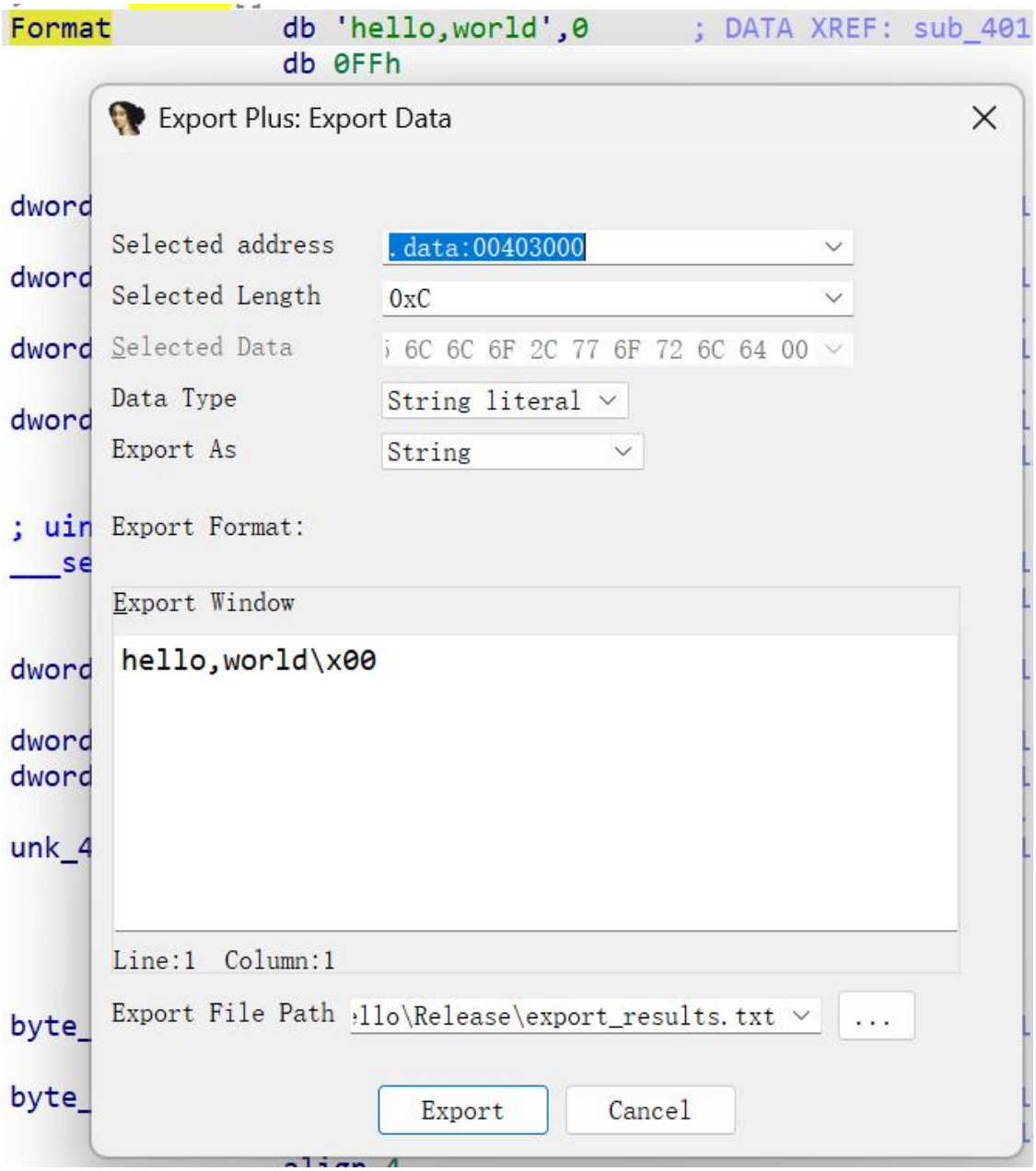


image-20260705193055147